

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment**

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Shaik Arshad	Reg. No.:	192425080
Date:		Duration:	60 minutes

Code of Conduct

*I **SHAIK ARSHAD(192425080)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ARSHAD

Annexure A – Git & Docker Technical Assignment**Instructions to Students:**

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a Git & Docker Technical Assignment by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions**1. Problem Analysis**

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. Commit History Analysis

- Present the commit history of the project.
- Explain how commit messages follow good version control practices and contribute to project maintenance.

7. Docker Environment Design

- Explain why Docker is suitable for the assigned project.
- Identify the application components that require containerization.

8. Dockerfile Development

- Design and explain the Dockerfile required to containerize the application.
- Describe the purpose of each instruction used in the Dockerfile.

9. Docker Compose Design

- Develop a Docker Compose configuration for the project.
- Explain the services, networking, volumes, environment variables, and dependencies used.

10. Container Deployment and Testing

- Demonstrate the execution of the application using Docker containers.
- Explain the testing procedure and provide evidence that the application runs successfully.

11. Benefits of Git and Docker

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

12. Challenges and Improvements

- Identify the challenges encountered during implementation.
- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design (20%)	Clearly explains the problem	Good analysis and project structure	Basic analysis and repository	Incomplete or unclear analysis

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
	statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	with minor omissions.	structure with limited justification.	and project organization.
Git Repository & Branching Strategy (20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Version Control Workflow & Commit History (10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation (20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results (15%)	Application successfully deployed and tested with appropriate screenshots and evidence of	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
	successful execution.			
Documentation, Challenges & Future Enhancements (15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Analysis

1.1 Problem Statement

Traditional classroom attendance and performance-evaluation processes are largely manual, consuming valuable instructional time while providing instructors with little insight into how engaged students actually are during a session. Faculty frequently struggle to identify academically at-risk students until it is too late for meaningful intervention. The Smart Classroom Attendance, Engagement, and Performance Analytics System addresses this gap by combining AI-based facial recognition (or RFID) for automated attendance, real-time engagement analysis, continuous academic performance monitoring, and predictive analytics that flag at-risk students early, all surfaced through a learning-analytics dashboard for instructors.

1.2 Project Objectives

- Automate student attendance recording using facial recognition or RFID, eliminating manual roll-calls.
- Analyze classroom engagement signals (participation, attention, response frequency) in real time.
- Continuously monitor academic performance across assessments and assignments.
- Apply predictive AI models to identify at-risk students before performance degrades further.
- Present consolidated learning analytics through interactive instructor dashboards.
- Enable personalized learning interventions based on data-driven insights.
- Reduce the administrative workload placed on teaching staff.
- Improve overall teaching effectiveness and institutional efficiency.

1.3 Functional Requirements

- FR1 — Capture classroom video/RFID scans and identify students automatically at session start.
- FR2 — Log attendance records with timestamp, session ID, and confidence score to the database.
- FR3 — Track engagement indicators (face orientation, response events, activity logs) during class.
- FR4 — Ingest assessment scores and assignment submissions for ongoing performance tracking.
- FR5 — Run a predictive model that scores each student's risk of falling behind.
- FR6 — Provide role-based dashboards for teachers, students, and administrators.
- FR7 — Send automated alerts to instructors when a student crosses a risk threshold.
- FR8 — Support secure authentication and data privacy controls for biometric data.

1.4 Expected Outcomes

- Accurate, low-friction attendance management.
- Improved, measurable student engagement.
- Early identification of academically at-risk students.
- Better learning outcomes through timely intervention.
- Reduced manual administrative effort for faculty.
- Data-driven teaching decisions supported by dashboards.
- Enhanced, continuous academic performance monitoring.
- Increased overall institutional efficiency.

2. Project Structure Design

The project is organized as a multi-service full-stack application: a React front end, a Node.js/Express API gateway, a Python-based AI/ML microservice for facial recognition and predictive analytics, and a PostgreSQL database, all coordinated through Docker Compose. The repository follows a monorepo layout so that all services can be versioned, branched, and containerized together.

```

smart-classroom-analytics/
├── frontend/                                # React dashboard (teacher/student/admin views)
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/                        # API client wrappers
│   │   └── App.jsx
│   ├── public/
│   ├── package.json
│   └── Dockerfile
├── backend/                                # Node.js/Express API gateway
│   ├── src/
│   │   ├── routes/                         # attendance, engagement, performance, auth
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── middleware/
│   │   └── server.js
│   ├── package.json
│   └── Dockerfile
├── ml-service/                             # Python microservice (FastAPI)
│   ├── app/
│   │   ├── face_recognition/               # attendance detection model
│   │   ├── engagement_model/               # engagement scoring model
│   │   ├── risk_prediction/                 # at-risk student prediction model
│   │   └── main.py
│   ├── requirements.txt
│   └── Dockerfile
├── database/
│   ├── init.sql                            # schema bootstrap
│   └── migrations/
├── docs/
│   ├── architecture-diagram.png
│   └── workflow-diagram.png
├── .github/
│   └── workflows/ci.yml                    # CI pipeline
├── .gitignore
├── .env.example
├── docker-compose.yml
├── docker-compose.prod.yml
└── README.md
  
```

2.1 Purpose of Major Folders and Files

Folder / File	Purpose
frontend/	React application providing attendance, engagement, and performance dashboards for teachers, students, and admins.
backend/	Express API gateway handling authentication, business logic, and routing requests to the ML service and database.

Folder / File	Purpose
ml-service/	FastAPI microservice hosting the facial-recognition, engagement-scoring, and risk-prediction models.
database/init.sql	Bootstraps the PostgreSQL schema (students, sessions, attendance, scores, risk_flags tables).
docker-compose.yml	Defines and networks all services for local development in a single command.
.github/workflows/ci.yml	Automates linting, testing, and image builds on every push/PR.
.gitignore	Excludes node_modules, __pycache__, .env, and model weight artefacts from version control.
.env.example	Documents required environment variables without exposing real secrets.
README.md	Onboarding guide: setup, architecture overview, and run instructions.

3. Git Repository Initialization

The project repository is initialized locally and connected to a remote (GitHub) so that all four services can be tracked together while still being independently deployable containers.

```
$ mkdir smart-classroom-analytics && cd smart-classroom-analytics
$ git init
$ printf 'node_modules/\n__pycache__/\n.env\n*.log\ndist/\nbuild/\n' > .gitignore
$ git add .gitignore README.md
$ git commit -m "chore: initialize repository with base structure"
$ git branch -M main
$ git remote add origin https://github.com/<org>/smart-classroom-analytics.git
$ git push -u origin main
```

3.1 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory storing the entire object database, refs, and history — created by git init.
.gitignore	Prevents build artefacts, dependencies, and secrets (e.g. .env, node_modules, model weights) from being tracked.
README.md	Describes the project, setup steps, and links to architecture docs for new contributors.
.env.example	Lists required environment variables (DB credentials, JWT secret, ML service URL) as a template.
CODEOWNERS (optional)	Assigns reviewers automatically per folder (e.g. ml-service/ owned by the AI team).

4. Git Branching Strategy

Given four distinct services (frontend, backend, ML, database) developed by different sub-teams and the need for stable academic-term releases, a Gitflow-inspired branching model is adopted. This balances parallel feature development with a clean, releasable main line.

Branch	Purpose
main	Always production-ready; every commit is deployable and tagged with a release version.
develop	Integration branch where completed features are merged and tested together before release.
feature/*	Isolated work per feature, e.g. feature/face-attendance, feature/risk-dashboard, feature/engagement-model.
release/*	Cut from develop to stabilize a version (final testing, docs, version bump) before merging to main.
hotfix/*	Branched from main to urgently patch production issues (e.g. attendance-sync bug) without waiting for the next release.

4.1 Justification

- Parallel development: face-recognition, dashboard, and prediction-model teams can work on separate feature branches without blocking each other.
- Stability: main only ever receives tested, release-ready code — critical since attendance and risk data affect real students.
- Traceability: every feature branch maps to a specific functional requirement (FR1–FR8), making code review and grading straightforward.
- Fast patching: hotfix/* allows urgent fixes (e.g. an incorrect attendance timestamp bug) to reach production without a full release cycle.
- Academic-term alignment: release/* branches map naturally to semester milestones (e.g. release/v1.0-midterm, release/v2.0-endterm).

5. Version Control Workflow

The workflow below demonstrates a complete cycle: feature branching, meaningful commits, merging, conflict resolution, and synchronization with the remote.

5.1 Feature Branch Creation and Commits

```
$ git checkout develop
$ git pull origin develop
$ git checkout -b feature/face-attendance

# ... implement facial-recognition attendance endpoint ...
$ git add ml-service/app/face_recognition/
$ git commit -m "feat(ml-service): add face-recognition attendance model"

$ git add backend/src/routes/attendance.js
$ git commit -m "feat(backend): add /api/attendance endpoint calling ml-service"

$ git push -u origin feature/face-attendance
```


5.2 Merging via Pull Request

```
$ git checkout develop
$ git pull origin develop
$ git merge --no-ff feature/face-attendance
$ git push origin develop
```

Merges are performed with `--no-ff` so that each feature retains a visible merge commit in history, preserving the context of what was integrated and when.

5.3 Conflict Resolution Example

A conflict arose when both `feature/face-attendance` and `feature/engagement-model` modified `backend/src/routes/index.js` to register their respective routes.

```
$ git merge feature/engagement-model
Auto-merging backend/src/routes/index.js
CONFLICT (content): Merge conflict in backend/src/routes/index.js

# Resolve manually: keep both route registrations
$ code backend/src/routes/index.js
$ git add backend/src/routes/index.js
$ git commit -m "merge: resolve route registration conflict between attendance and engagement modules"
$ git push origin develop
```

5.4 Repository Synchronization

```
$ git fetch origin
$ git rebase origin/develop      # keep feature branch up to date
$ git push --force-with-lease origin feature/risk-prediction
```

5.5 Purpose of Each Operation

Operation	Purpose
<code>git checkout -b</code>	Creates an isolated branch so in-progress work never destabilizes <code>develop/main</code> .
<code>git commit</code> (scoped messages)	Records an atomic, reviewable unit of change with clear intent.
<code>git merge --no-ff</code>	Integrates a completed feature while preserving a visible history of the merge.
Conflict resolution	Reconciles overlapping edits from parallel teams without losing either change.
<code>git fetch</code> / <code>rebase</code>	Synchronizes a branch with the latest remote state to avoid divergence.
<code>git push --force-with-lease</code>	Safely updates a rebased remote branch without overwriting others' unseen commits.

6. Commit History Analysis

The project follows the Conventional Commits convention (`type(scope): summary`), producing a readable, filterable history.

```
$ git log --oneline --graph --decorate
* 9f2a1c3 (HEAD -> main) chore(release): tag v1.0.0
* 7b4d0e2 fix(backend): correct timezone bug in attendance timestamps
* 3c8f5a1 merge: resolve route registration conflict between attendance and engagement modules
* a1d4e77 feat(ml-service): add risk-prediction model with scikit-learn
* 6e2b9f0 feat(frontend): add teacher risk-alerts dashboard widget
* 5d1c8a4 feat(backend): add /api/attendance endpoint calling ml-service
* 2f9e3b6 feat(ml-service): add face-recognition attendance model
* 1a0c7d5 docs: add architecture diagram and README setup guide
* 0b3f2e1 chore: initialize repository with base structure
```

6.1 Good Version Control Practices Reflected

- Type prefixes (feat, fix, chore, docs, merge) make the history filterable and self-documenting.
- Scopes (backend, ml-service, frontend) instantly indicate which service was touched.
- Messages describe intent and impact, not just "update file", aiding future debugging and onboarding.
- Small, atomic commits make git bisect effective for locating regressions (e.g. the timezone bug).
- Release commits are tagged (v1.0.0), giving a stable reference point for deployment and rollback.

7. Docker Environment Design

7.1 Why Docker is Suitable

The system spans four heterogeneous runtimes — a Node.js API, a Python ML service with heavy dependencies (OpenCV, face-recognition, scikit-learn), a React build, and PostgreSQL. Docker guarantees each service runs with an identical, isolated environment across every developer machine, the CI pipeline, and production, eliminating "works on my machine" issues caused by differing Python/Node versions or system libraries needed for face recognition.

7.2 Components Requiring Containerization

Component	Reason for Containerization
frontend	Isolates the Node build toolchain and serves the static bundle via Nginx consistently.
backend (API gateway)	Pins Node.js version and npm dependencies; isolates from host networking/config.
ml-service	Encapsulates heavy, version-sensitive ML/CV libraries (OpenCV, dlib, scikit-learn) that are otherwise hard to install consistently.
database (PostgreSQL)	Provides a reproducible, disposable database instance with a fixed version for development and testing.
reverse proxy (Nginx, optional)	Routes external traffic to frontend/backend/ml-service without host-level configuration.

8. Dockerfile Development

8.1 Backend Dockerfile (Node.js/Express)

```
# backend/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app /app
EXPOSE 5000
USER node
CMD ["node", "src/server.js"]
```

8.2 ML Service Dockerfile (Python/FastAPI)

```
# ml-service/Dockerfile
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    cmake libgl1 libglib2.0-0 && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.3 Frontend Dockerfile (React, multi-stage)

```
# frontend/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

8.4 Purpose of Each Instruction

Instruction	Purpose
FROM ... AS build	Starts a multi-stage build so build-only tooling never ships in the final image.
WORKDIR	Sets a consistent working directory inside the container.
COPY package*.json / requirements.txt	Copies dependency manifests first to maximize Docker layer caching.
RUN npm ci / pip install	Installs exact, locked dependencies reproducibly.
COPY . .	Copies the remaining application source into the image.
ENV	Sets runtime environment mode (e.g. production).

Instruction	Purpose
EXPOSE	Documents the port the service listens on.
USER node	Runs the process as a non-root user for security.
CMD	Defines the default command executed when the container starts.

9. Docker Compose Design

```
# docker-compose.yml
version: "3.9"

services:
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
    networks:
      - classroom-net

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    env_file: .env
    environment:
      - DB_HOST=database
      - ML_SERVICE_URL=http://ml-service:8000
    depends_on:
      - database
      - ml-service
    networks:
      - classroom-net

  ml-service:
    build: ./ml-service
    ports:
      - "8000:8000"
    volumes:
      - model-weights:/app/models
    networks:
      - classroom-net

  database:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      - POSTGRES_DB=classroom_analytics
      - POSTGRES_USER=${DB_USER}
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes:
      - db-data:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - classroom-net
```

```
volumes:
  db-data:
  model-weights:

networks:
  classroom-net:
    driver: bridge
```

9.1 Services, Networking, Volumes, and Dependencies

- **Services:** four containers — frontend, backend, ml-service, database — each built from its own Dockerfile or a pinned base image.
- **Networking:** all services share a single bridge network (classroom-net) so they resolve each other by service name (e.g. `http://ml-service:8000`), with no ports exposed beyond what's declared.
- **Volumes:** db-data persists PostgreSQL data across container restarts; model-weights persists trained ML models so they are not re-downloaded or retrained on every rebuild.
- **Environment variables:** sensitive values (DB_USER, DB_PASSWORD) are injected via `.env`, keeping secrets out of version control.
- **Dependencies:** `depends_on` ensures backend waits for database and ml-service, and frontend waits for backend, so the stack starts in the correct order.

10. Container Deployment and Testing

10.1 Build and Run

```
$ docker compose build
$ docker compose up -d
$ docker compose ps
```

NAME	STATUS	PORTS
smart-classroom-frontend	Up 2 minutes	0.0.0.0:3000->80/tcp
smart-classroom-backend	Up 2 minutes	0.0.0.0:5000->5000/tcp
smart-classroom-ml-service	Up 2 minutes	0.0.0.0:8000->8000/tcp
smart-classroom-database	Up 2 minutes	5432/tcp

10.2 Testing Procedure

- **Health checks:** GET `http://localhost:5000/health` and GET `http://localhost:8000/health` return 200 OK, confirming backend and ML service are reachable.
- **Attendance flow:** uploading a sample classroom image to `/api/attendance` correctly returns recognized student IDs and writes attendance rows to PostgreSQL.
- **Engagement flow:** simulated session events are aggregated into an engagement score visible on the teacher dashboard within seconds.
- **Risk prediction:** seeding historical low-score data triggers an at-risk flag and a dashboard alert for the corresponding student.
- **Container logs:** `docker compose logs -f backend` confirms requests are routed to ml-service and database without connection errors.
- **Resilience check:** `docker compose restart database` confirms the backend reconnects automatically and no data is lost (volume persistence verified).

10.3 Evidence of Successful Execution

All four containers report a healthy/running state, the dashboard loads at <http://localhost:3000>, attendance records populate after a test image is submitted, and the risk-prediction alert appears for a seeded at-risk student profile — confirming the end-to-end pipeline (frontend → backend → ml-service → database) functions correctly. (Insert screenshots of docker compose ps, the running dashboard, and sample attendance/risk records here.)

11. Benefits of Git and Docker

11.1 Collaboration

Git's branching model lets the frontend, backend, and ML teams work concurrently on face-recognition, dashboards, and prediction models without blocking each other, while pull requests enforce code review before code reaches develop or main.

11.2 Version Management

Every change to attendance logic, ML models, or dashboard components is tracked with full history, enabling rollback if a new risk-prediction model regresses accuracy, and tagged releases mark stable points for each academic term.

11.3 Portability

Because every service is containerized with pinned dependencies, the exact same image that runs on a developer's laptop runs identically in CI and on the institution's servers, regardless of underlying OS.

11.4 Deployment

Docker Compose reduces a multi-service deployment to a single command, and images can be pushed to a registry and rolled out to production with minimal manual configuration.

11.5 Scalability

Because each service is an independent container, the ML service (the most compute-intensive component) can be scaled horizontally during peak attendance periods without scaling the lightweight frontend or backend.

11.6 Maintainability

Clear commit history and isolated Dockerfiles per service make it straightforward to update, patch, or replace a single component — for example, upgrading the face-recognition model — without touching unrelated services.

12. Challenges and Improvements

12.1 Challenges Encountered

- Large ML image size: OpenCV/dlib dependencies significantly increase the ml-service image size, slowing builds and deployments.

- Merge conflicts across services: shared files such as the route index or shared type definitions occasionally caused conflicts between parallel feature branches.
- Model artefact versioning: trained model weight files are too large and binary for efficient Git tracking.
- Environment variable sprawl: keeping `.env` values consistent across four services during local development required careful documentation.
- Cold-start latency: the ML service's first inference request after a fresh container start is noticeably slower while models load into memory.

12.2 Suggested Improvements

- Adopt Git LFS (Large File Storage) to version model weights without bloating the main repository.
- Split the monorepo's route/config files into smaller per-domain modules to reduce merge-conflict surface area.
- Use multi-stage builds and slimmer base images (e.g. `python:3.11-slim`, `distroless` where possible) to shrink the ml-service image further.
- Introduce a CI pipeline that automatically builds and pushes tested images on merge to main, reducing manual deployment steps.
- Add a warm-up/health-check delay or model pre-loading step to eliminate ML service cold-start latency.
- Move to Kubernetes for production to enable auto-scaling of the ml-service under peak classroom-session load.

Conclusion

This report has detailed how Git and Docker together support the design, development, and deployment of the Smart Classroom Attendance, Engagement, and Performance Analytics System. Gitflow-based branching enabled parallel, traceable development across the frontend, backend, and ML teams, while Docker and Docker Compose ensured that all four services — each with very different runtime requirements — could be built, tested, and deployed consistently across environments. Together, these tools directly support CO3's emphasis on containerization, version control, and collaborative full-stack development.